

PYTHON FOR EMBEDDED & AUTOMATION FRAMEWORK DESIGN

A Rigorous, First-Principles Hardware-in-the-Loop Test Suite
Curriculum

Duration: 12 Weeks | **Level:** Advanced Automation

Mentor: Senior Automation Architect & Coach

Workspace Root: /home/siva/sivaramireddy/Project1/python-automation-curriculum/

Python for Embedded Systems & Automation Framework Design: Onboarding Guide

Welcome to the **Python for Embedded Systems & Automation Framework Design Master Plan**. This curriculum is designed by a Senior Systems Coach to transition engineering students into elite automation architects capable of constructing multi-threaded, hardware-in-the-loop (HIL) test suites and CI/CD pipelines.

In modern hardware teams, Python is the glue. It does not replace microcontroller microcode, but instead wraps around it, orchestrating compiler pipelines, running physical testing instruments (oscilloscopes, load units), logging high-speed sensor streams, and auditing binary compilation maps.

Curriculum Workspace Layout

Your training environment is structured cleanly inside the `python-automation-curriculum/` folder:

```
python-automation-curriculum/
├── README.md                # This Onboarding & Setup Portal
├── python-trainer-rules.md  # Strict Python style laws, design patterns, and
type policies
├── 12-week-python-roadmap.md # Master chronological milestones timeline and
summary
├── python-milestone-scorecard.md # Gatekeeper assignments and grading scorecards
├── plans/                  # Deep-dive weekly execution plans
│   ├── week-01-python-plan.md # Week 01: Host Automation & CLI wrapping
│   ├── week-02-python-plan.md # Week 02: Hardware Scripting & Serial
Communications
│   │   ...
│   └── week-12-python-plan.md # Week 12: Production-Grade Capstone HIL Framework
Project
└── build_pdf_python.py      # Automated compilation tool for PDF generation
```

Environment Setup & Toolchain Index

To ensure reproducibility, you will manage dependencies using isolated virtual environments. Standard global pip installs are banned.

1. Ubuntu Linux Prereqs Installation

Ensure your host machine has Python 3.10+ and the required system packages installed:

```
sudo apt update && sudo apt install -y \  
python3-pip \  
python3-venv \  
python3-dev \  
libusb-1.0-0-dev \  
weasyprint
```

2. Environment Isolation Setup

We will use Python's built-in `venv` module for environment isolation. Inside the curriculum root, run:

```
# Create virtual environment  
python3 -m venv .venv  
  
# Activate virtual environment  
source .venv/bin/activate  
  
# Upgrade packaging tools  
pip install --upgrade pip setuptools wheel
```

3. Core Automation Packages Checklist

Install the mandatory systems-level Python libraries within your virtual environment:

```
pip install \  
    pytest \  
    mypy \  
    pyserial \  
    pyelftools \  
    pyvisa \  
    pandas \  
    matplotlib \  
    click \  
    black \  
    ruff
```

Static Analysis & Formatting Standards

All automation code developed in this curriculum must be audited defensively using static checkers:

1. **Strict Type Hinting** (`mypy`): All function parameters and returns must carry explicit type annotations. No untyped signatures allowed. `bash mypy --strict src/`
 2. **Deterministic Formatting** (`black`): Automate visual consistency by running the black code formatter. `bash black src/`
 3. **Advanced Linting** (`ruff`): Scan for style issues, dead code, and performance traps. `bash ruff check src/`
-

Let's Begin

Proceed next to read the [python-trainer-rules.md](#) to understand the strict quality laws governing your submissions, and then trace your journey in [12-week-python-roadmap.md](#)!

Professional Coach: Python Automation

operating Rules & Laws

As an automation architect and Senior Systems Coach, my role is to hold you to the absolute highest software engineering standards. Below are the **strict rules and laws** governing all Python code submissions during this 12-week intensive Automation Framework track.

Any violation of these rules results in an automatic **FAIL** for that week's assignment. No exceptions.

Rule 1: The "Strict-Type" Directive

All Python files must incorporate explicit C-like static type annotations. * **The Law:** You must type all function inputs, outputs, and class member variables. The `typing` module (`List`, `Dict`, `Union`, `Callable`) must be leveraged for complex mappings. * **The Verification:** Every submission is audited using **Mypy**: `bash mypy --strict src/`. If Mypy emits a single warning, typing omission, or dynamic type conversion alert, your code is rejected immediately without further review. * **The Logic:** Dynamic typing is a primary source of runtime bugs in large automation frameworks. Explicit static type hints act as compile-time documentation, preventing hardware command parameter mismatches.

Rule 2: Strict Hardware Connection & Thread Safety

When automation scripts interface with physical instruments (VISA, Serial, TCP), connections must be guarded. * **The Law:** All hardware connections must implement **Context Managers** (`__enter__` and `__exit__`) to guarantee that ports and sockets are closed cleanly on errors. * **Concurrency Law:** When reading telemetry streams inside parallel threads, shared objects must be accessed safely via thread-safe structures (`queue.Queue`) or guarded by execution Locks (`threading.Lock`).

Rule 3: Mandatory Design Patterns for Hardware Control

To write maintainable HIL test suites, you must adhere to three core object-oriented design patterns:

Pattern Name	Application	Rationale
Singleton Handle	Dedicated connection drivers (e.g. <code>VISAConnectionManager</code>).	Prevents open-port lock conflicts by ensuring only a single dynamic connection handler instance exists for each instrument.
Device Object (DOM)	Mapping raw register bytes to high-level device classes.	Decouples raw hardware commands (e.g., <code>write("*RST")</code>) from logical assertions (e.g., <code>scope.reset()</code>).
Factory Pattern	Dynamic test runner hardware target variants.	Allows test frameworks to swap mock hardware objects with physical hardware dynamically based on test configurations.

Rule 4: pytest Hardening & Fixture Isolation

- **The Law:** Raw test functions must be stateless. All setups (powering up instrument) and teardowns (graceful shutdown) must occur within scoped **pytest fixtures**.
- **Global States Forbidden:** Global variables used to store test states or hardware handles are strictly banned. Everything must be passed dynamically via fixture dependency injection.

Coach's Grading Rubric

Each week, your coach evaluates your deliverables out of a 10-point scale:

0 to 6 points: Rejected / Remediation Required

- **Criteria:** Static typing warnings present under `mypy --strict`, unhandled hardware resource leaks, global variables in test states, or missing teardowns in fixtures.
- **Coach's Directive:** Resubmit within 48 hours after applying required design patterns and type hardening.

7 to 8 points: Passed

- **Criteria:** Zero Mypy errors, clean pytest test runs, proper context managers. However, there are minor layout, lack of parameterization, or sub-optimal threading designs.
- **Coach's Directive:** Approved to advance, but styling fixes must be applied before the next milestone audit.

 9 to 10 points: Passed with Distinction

- **Criteria:** Flawless typed architecture, 100% unit/integration coverage, elegant design pattern implementation, and completion of the complex **Stretch Task**.
- **Coach's Directive:** Approved to advance with commendation.

Master Curriculum Roadmap: Python for Embedded Systems & Automation

This 12-week curriculum is a fast-paced, high-intensity software engineering program. It is designed to take you from a basic Python programmer into an elite test automation architect who builds robust Hardware-in-the-Loop (HIL) test suites, CI/CD pipelines, and binary parsing scripts.



Chronological Milestones Overview

```
graph TD
    Phase1[Milestone 1: Embedded Python & Hardware Scripting<br/>Weeks 1-3] --> Phase2[Milestone 2: Test Automation & HIL Testing<br/>Weeks 4-6]
    Phase2 --> Phase3[Milestone 3: Advanced Automation & Binary Auditing<br/>Weeks 7-9]
    Phase3 --> Phase4[Milestone 4: Enterprise Framework & CI/CD<br/>Weeks 10-12]
```



Chronological Week-by-Week Breakdown

◆ Milestone 1: Embedded Python & Hardware Scripting

Focus: Command-line wrapper tools, parsing binary frames over serial pipelines, and flashing and controlling Edge microcontrollers (MicroPython).

• Week 01: Host Automation & CLI wrapping

- Theme: Subprocess redirection, defensive input execution, argument parsing, automated flashes.
- Assignment: Build a robust CLI compiling and flashing wrapper that schedules compiler pipelines dynamically.

• Week 02: Hardware Scripting & Serial Communications

- Theme: PySerial architectures, unbuffered byte reading, binary packing (`struct`), CRC validation.
- Assignment: Write a packet serialization/deserialization telemetry reader parsing raw USB streams.

• Week 03: MicroPython & Microcontroller Prototyping

- Theme: MicroPython execution context, GPIO, ADC, PWM registers control, interrupt safety, USB REPL.

- Assignment: Construct an edge temperature logger running on simulated MCU hardware.
 - Gatekeeper Check: **Milestone 1 Core Gatekeeper Assignment.**
-

♦ Milestone 2: Test Automation & Hardware-in-the-Loop (HIL)

Focus: Lab instrumentation wrapping, SCPI/VISA standards, structural pytest automation, and fault-injection HIL setups.

• Week 04: Instrumentation Control (PyVISA & SCPI)

- Theme: Instrument control interfaces, PyVISA VISA resource maps, SCPI string tables, Raw TCP sockets.
- Assignment: Design a complete, simulated Programmable Power Supply VISA driver wrapper.

• Week 05: Test Framework Architecture (pytest & Fixtures)

- Theme: Setup/teardown isolation fixtures, dynamic discovery, test parametrizations, timeouts and retries.
- Assignment: Write a multi-instrument hardware test suite using robust pytest dependencies.

• Week 06: Hardware-in-the-Loop (HIL) & Fault Injection

- Theme: Relays control, automated fault injection, loop verification, asynchronous failure monitors.
 - Assignment: Build an automated fault-injection test system simulating real hardware failures.
 - Gatekeeper Check: **Milestone 2 Core Gatekeeper Assignment.**
-

♦ Milestone 3: Advanced Automation & Binary Auditing

Focus: Parsing compiled firmware images, high-performance telemetry data logging, and metrics reporting dashboards.

• Week 07: Firmware Analysis & Binary Parsing

- Theme: ELF binary parsing (`pyelftools`), compiler memory mapping, symbol address resolution.
- Assignment: Write a static memory auditor checking ELF sections sizes and warning on stack overlaps.

• Week 08: High-Performance Telemetry Data Logging

- Theme: Thread-safe serial readers, data queues, streaming file storage (CSV, Parquet, Pandas integration).
- Assignment: Design a multi-threaded telemetry streaming data-logger that processes high-frequency feeds.

• Week 09: Dynamic CLI Tools & Metrics Reporting

- Theme: Interactive CLI builders (`click`), automated report generators (PDF, HTML reports generation).
- Assignment: Build a CLI metrics generator generating professional PDF test reports dynamically.

- Gatekeeper Check: **Milestone 3 Core Gatekeeper Assignment.**
-

♦ **Milestone 4: Enterprise Framework & CI/CD**

Focus: Design patterns, static type audits, Jenkins/GitHub Actions integrations, and graduation capstone execution.

- **Week 10: Python Code Quality, Typing, & Design Patterns**

- Theme: Mypy static type hints, design patterns (Singleton, Factory, Device wrappers), Black/Ruff checkers.
- Assignment: Refactor a legacy, untyped testing framework into a fully-typed, object-oriented framework.

- **Week 11: CI/CD Pipelines & Execution Orchestration**

- Theme: Automated pipelines orchestration, parsing JUnit XML test outputs, Slack/email reports integrations.
- Assignment: Construct an automated Jenkinsfile/GitHub Actions workflow orchestrating HIL tests on git push.

- **Week 12: Production-Grade Capstone HIL Framework Project**

- Theme: Multi-threaded frameworks assembly, error diagnostics, final coaching portfolio presentations.
 - Assignment: Design a complete, modular, fully-typed, and multi-threaded **Hardware-in-the-Loop test automation framework** with zero errors.
 - Gatekeeper Check: **Milestone 4 Final Graduation Audit.**
-

Advancement Directive

Before beginning, review the [python-milestone-scorecard.md](#) to understand the requirements of the Milestone Gates you must clear to graduate this curriculum!

Professional Python Automation: Milestone Scorecard

To progress through the curriculum, the student must successfully clear the **Gatekeeper Assignment** at the end of each milestone. Standard curves are banned: these assessments are scored on an industrial **Pass/Fail** basis against a strict engineering rubric.

Milestone 1 Gatekeeper: Dynamic MicroPython Telemetry Node

- **Checkpoint:** End of Week 03 (Milestone 1)
- **Goal:** Verify compiler subprocess wrapping, pySerial communications, MicroPython REPL scripting, and packet serialization checks.

Core Specifications

1. Write an automated host-side controller in Python that connects to an edge microcontroller running MicroPython over serial USB.
2. The host must wrap the `ampy` or `rshell` command-line tools to dynamically flash the MicroPython telemetry code.
3. Once flashed, the host must connect over PySerial, parse raw binary streams packet-by-packet (`struct` unpack), and validate each payload using a 16-bit Cyclic Redundancy Check (CRC-16).
4. Code must be 100% type-annotated, checked with `mypy --strict`, and formatted cleanly with `black`.

Evaluation Scoring Rubric

Evaluation Metric	Passing Criteria (Must Met)	Triggers Automatic Failure (FAIL)
Typing & Formatting	<code>mypy --strict</code> passes with 0 warnings; Black formatter returns zero style shifts.	Single untyped parameter, use of <code>Any</code> type, or formatting check errors.
Telemetry Parsing	Unpacks raw byte streams successfully; detects CRC checksum mismatches.	Buffer overflow exceptions, missing CRC checks, or unclosed serial ports.
CLI Wrapper	Subprocess flashing executes with strict timeouts and grabs execution errors.	Hanging subprocess commands without timeout configurations.

Milestone 2 Gatekeeper: Instrumented Hardware-in-the-Loop Test Suite

- **Checkpoint:** End of Week 06 (Milestone 2)
- **Goal:** Verify VISA instrumentation wrapping, pytest fixtures architectures, parameterizations, and automated fault-injection HIL scripting.

Core Specifications

1. Wrap simulated lab instruments (Power Supply, Multimeter, Oscilloscope) using the **Singleton pattern** for raw VISA connection handles.
2. Implement **Device Object Models (DOM)** to provide clean APIs (e.g. `power_supply.set_voltage(5.0)`).
3. Build a pytest-driven automated testing harness containing:
4. A class-scoped `hardware_setup` fixture establishing VISA connections and configuring standard safe voltage ceilings.
5. Functional test cases that inject faults (e.g. commanding the power supply to brownout) and assert that the device behaves defensively.
6. Manage setups and teardowns safely within fixtures, ensuring instrument power rails are shut off in all exit paths.

Evaluation Scoring Rubric

Evaluation Metric	Passing Criteria (Must Met)	Triggers Automatic Failure (FAIL)
Resource Safety	Instrument connections closed in all exit paths using context managers or fixture teardowns.	Unclosed sockets, lingering VISA sessions, or failing to cut power on failure.
Fixture Discipline	All setup/teardowns governed via dynamic fixture dependency injections; 0 global states.	Using global variables to store test states or connection pointers.
Parametrization	Test cases utilize <code>@pytest.mark.parametrize</code> to scale verification across multiple voltage bounds.	Redundant copy-pasted test cases for individual values.

Milestone 3 Gatekeeper: ELF Firmware Analyzer & Telemetry Plotter

- **Checkpoint:** End of Week 09 (Milestone 3)
- **Goal:** Verify ELF binary structural auditing (`pyelftools`), multi-threaded telemetry logging, and CLI metrics reporting.

Core Specifications

1. Write a firmware static memory analysis tool using `pyelftools` that parses a compiled ELF image. The tool must:
2. Audit memory sizes (`.text`, `.data`, `.bss`) and flag alerts if allocation limits are exceeded.
3. Retrieve symbol address tables and check for potential stack-to-heap collisions.
4. Build a multi-threaded telemetry streaming data-logger. Use `threading.Thread` and a thread-safe `queue.Queue` to stream binary packet inputs to disk, avoiding data losses.
5. Design a CLI interface using `click` that compiles these metrics dynamically and exports a beautifully formatted PDF report containing performance graphs (using Matplotlib).

Evaluation Scoring Rubric

Evaluation Metric	Passing Criteria (Must Met)	Triggers Automatic Failure (FAIL)
Binary Analysis	Successfully extracts symbol offsets and calculates size segments of ELF binaries.	Crashing on malformed ELF structures, incorrect pointer math offsets calculations.
Thread Safety	Telemetry queue reads are thread-safe and non-blocking, avoiding race conditions or lockouts.	Data losses on high-speed queues, unhandled thread exceptions, or locked threads.
CLI & Reports	Click parser handles invalid arguments gracefully; PDF generates correct Matplotlib graphs.	CLI crash on bad inputs; raw stdout prints without structured PDF outputs.

Milestone 4 Gatekeeper: Enterprise CI/CD HIL Framework

- **Checkpoint:** End of Week 12 (Milestone 4 / Graduation)
- **Goal:** Verify complete type safety, OOP design patterns, CI/CD automated configurations, and graduation capstone execution.

Core Specifications

1. Design a comprehensive, modular **Hardware-in-the-Loop Test Automation Framework** from scratch.
2. Structure the codebase using clean OOP patterns: Factory pattern for target boards selection, Device Object abstractions, Singleton connection managers.
3. Integrate 100% strict static typing, validated using `mypy --strict`.
4. Build a continuous integration workflow configuration (Jenkinsfile or GitHub Actions YAML) that:
5. Automatically executes the test suite inside your virtual environment.
6. Generates a JUnit XML metrics report.
7. Rejects merges if test coverage falls below 90% or if static audits fail.

Evaluation Scoring Rubric

Evaluation Metric	Passing Criteria (Must Met)	Triggers Automatic Failure (FAIL)
Framework Design	Code utilizes clean DOM and Factory designs; 0 typing omissions or Any casts under Mypy.	Spaghetti structures, raw VISA commands inside test cases, or Mypy type-check errors.
Pipeline Integration	CI/CD pipeline triggers test runs on commits, generates HTML reports, and parses JUnit output.	Pipeline script failure, missing reports, or failing to block merges on test failures.
Graduation Portfolio	Complete portfolio review presenting test architectures, 100% type-checked framework, and CI/CD results.	Incomplete code documentation, missing test cases, or failing type checks.

Week 01: Host Automation & CLI Wrapping

Weekly Goal

Master Python's system scripting boundaries, control compiler and hardware flashing CLI tools via the `subprocess` module, handle path manipulations cleanly, and construct robust command-line argument parsers.

Weekly Outcome

By the end of this week, you will be able to launch and monitor system processes defensively from Python, capture and parse stdout/stderr streams without blocking thread execution, handle paths portably across systems using `pathlib`, and build type-safe CLI tools.

Session Breakdown

Monday: The Subprocess Execution Model

- **Conceptual Focus:** Interfacing Python with the operating system shell.
- Why `os.system` is deprecated and dangerous (vulnerable to shell-injection security flaws).
- The `subprocess` module as the modern standard.
- Running blocking processes using `subprocess.run`.
- Running asynchronous non-blocking processes using `subprocess.Popen`.
- **Hands-on Experiment:** Execute a system directory listing safely: `python import subprocess`

```
# Run list directory command result = subprocess.run( ["ls", "-la"], capture_output=True, text=True, check=True ) print("STDOUT output:") print(result.stdout)``
```

Tuesday: Defensive Stream Redirects & Timeouts

- **Conceptual Focus:** Preventing hung processes.
- In automation frameworks, calling a flashing tool that hangs indefinitely can freeze entire CI/CD pipelines.
- **The Law:** All subprocess calls must enforce explicit execution timeouts.
- Capturing error outputs by mapping `stderr` to `stdout` pipes.
- **Hands-on Exercise:** Catch execution timeouts and parse shell errors dynamically: `python import subprocess`

```
try: # Simulate a process that hangs (sleep 10) but set a 2 second timeout subprocess.run(["sleep",
"10"], timeout=2) except subprocess.TimeoutExpired as e: print(f"Process timed out safely! Details:
{e}") ``
```

Wednesday: Path Operations (`pathlib`)

- **Conceptual Focus:** The portability trap.
- Windows uses backslashes (`\`) for file paths; Linux uses forward slashes (`/`). Hardcoding strings breaks code cross-platform.
- **The Solution:** Python's modern `pathlib` library representing paths as abstract system objects rather than strings.
- **Hands-on Experiment:** Manipulate directory workspaces portably: `python from pathlib import Path`

```
# Get absolute path of current directory curr_dir = Path(".").resolve() print(f"Absolute Workspace:
{curr_dir}")
```

```
# Join paths and create folders build_dir = curr_dir / "build" / "output" build_dir.mkdir(parents=True,
exist_ok=True) print(f"Build output directory successfully created: {build_dir.exists()}") ``
```

Thursday: Argument Parsing (`argparse`)

- **Conceptual Focus:** Designing structured user interfaces.
- Writing custom command-line interfaces (CLIs).
- Validating argument data types, setting defaults, and generating help manuals.
- **Hands-on Exercise:** Build a basic CLI configuration script: `python import argparse`

```
def main() -> None: parser = argparse.ArgumentParser(description="Compiler automation tool.")
parser.add_argument("-t", "--target", required=True, help="MCU target string (e.g. esp32, rp2040)")
parser.add_argument("-v", "--voltage", type=float, default=3.3, help="Input voltage check value")
args = parser.parse_args() print(f"Target: {args.target}, Voltage check: {args.voltage}V")
```

```
if name == "main": main() ``
```

Friday: Configuration Files (YAML / JSON)

- **Conceptual Focus:** Bypassing hardcoded configuration scripts.
 - Reading and validating test configuration parameters dynamically from files.
-

Hands-On Assignment: Automated GCC & Flashing Wrapper

Design and build a robust Python automation command-line tool named `flash_build.py` that automates compiler and flashing pipelines.

1. Requirements

- Provide a CLI interface that accepts: `--src` (path to C source folder), `--bin` (path to output binary), and `--baud` (baudrate for flashing).
- Using the `subprocess` module:
- Compile all C files in the target folder using `gcc -Wall -Wextra -Werror` inside a 5-second execution timeout.
- If compilation fails, parse the stderr output and highlight compiler error lines.
- If compilation succeeds, simulate a hardware flashing process by calling a mockup flashing script, handling timeout exceptions cleanly.
- All code must be strictly type-annotated and pass `mypy --strict`.

2. File Deliverables

- `flash_build.py` : Main CLI tool code.
- `test_workspace/` : A folder containing both a clean-compiling C file and a broken C file for test scenarios.

Deliverables Checklist

- [] `flash_build.py` (Mypy type-hinted, zero warnings)
- [] `test_workspace/` (C compile verification targets)
- [] `requirements.txt` (Environment libraries dependencies)

? Self-Check Questions

1. Why is passing arguments as a list (e.g. `["ls", "-l"]`) in `subprocess` significantly safer than passing a single string (`"ls -l"`)?
2. What happens to a subprocess that exceeds its timeout threshold, and how do you recover?
3. How does `pathlib.Path` resolve path separators differences when running on a Windows vs Linux machine?
4. Explain the difference between blocking `subprocess.run()` and non-blocking `subprocess.Popen()`.

5. Why are hardcoded configurations inside scripts considered a major design flaw in testing frameworks?

Stretch Task

Modify your flashing wrapper to support **parallel compilations**. Implement a multi-processed compile manager that calls parallel compilers dynamically to compile multiple C files concurrently, merging outputs safely.

Motivation Checkpoint

In silicon development teams, automated scripts compile and flash firmware thousands of times a day across massive server farms. A single unhandled timeout or path corruption can freeze testing arrays and waste thousands of development dollars. Writing robust, defensive system scripts is the first step of automation architecture.

Trainer Review Rules

- Verify type-safety: Executing `mypy --strict flash_build.py` must complete with 0 alerts.
- Verify subprocess handling: Executing a compiler call that fails must be intercepted, and the clean terminal output must display only the compiler's syntax errors without throwing Python traceback crashes.

Week 02: Hardware Scripting & Serial Communications

Weekly Goal

Master serial communication protocols from the host side using PySerial, read and write raw byte streams defensively, and serialize/deserialize complex data structures using Python's `struct` module.

Weekly Outcome

By the end of this week, you will be able to establish robust serial USB connections, manage serial stream buffer read loops, map raw incoming bytes to high-level Python dataclasses using `struct.unpack`, and validate data integrity using CRC-16 checks.

17 July Session Breakdown

Monday: The PySerial Communications Layer

- **Conceptual Focus:** Interfacing with serial channels.
- Configuring port names, baudrates, timeouts, parity, and stop bits.
- **The Trap:** Blocking read operations without timeouts. Your read loops must always execute defensively using non-zero read timeouts.
- Clearing input/output buffers: `reset_input_buffer()`, `reset_output_buffer()`.
- **Hands-on Experiment:** Establish and open a serial connection: `python import serial`

```
# Configure serial port defensively try: with serial.Serial(port="/dev/ttyUSB0", baudrate=115200,
timeout=1.0) as ser: print(f"Serial port {ser.name} opened successfully!") # Perform safe write
ser.write(b"PING\n") except serial.SerialException as e: print(f"Connection failed: {e}") ``
```

Tuesday: The `struct` Packing Module

- **Conceptual Focus:** Binary conversions in Python.
- Microcontrollers output data as raw binary bytes. Python must map these bytes to floats, ints, or chars.
- **The Solution:** Python's built-in `struct` module.

- Code formatting codes: `f` (float), `H` (16-bit unsigned int), `B` (8-bit unsigned int), `<` (force Little-Endian), `>` (force Big-Endian).

- **Hands-on Exercise:** Pack and unpack telemetry data: `python import struct`

```
# Pack: 8-bit ID (1), 32-bit float temp (25.5), 16-bit counter (100) # Little-Endian format: '< B f H'
binary_packet = struct.pack("<BfH", 1, 25.5, 100) print(f"Packed Bytes: {binary_packet.hex().upper()}")
```

```
# Unpack device_id, temp, counter = struct.unpack(" ID: {device_id}, Temp: {temp}, Counter: {counter}") ``
```

Wednesday: Telemetry Byte Stream Parsing

- **Conceptual Focus:** Finding packet boundaries.
 - When reading raw byte streams over serial, data is not delivered in neat, separate packets. It arrives as a continuous stream of random bytes.
 - **Framing:** Using start-of-packet (`SOP`) and end-of-packet (`EOP`) byte codes (e.g. `0x02` and `0x03`) to identify valid packet blocks.
 - **Hands-on Exercise:** Write a defensive parser loop that scans incoming bytes and extracts complete packet frames.
-

Thursday: CRC-16 Checksums & Data Integrity

- **Conceptual Focus:** Validating hardware packet transmissions.
 - Serial connections are noisy. EMI interference can flip bits.
 - Using **Cyclic Redundancy Checks (CRC-16)** to detect packet corruptions.
 - **Hands-on Exercise:** Write a pure Python CRC-16 calculation function to validate packet frames before parsing.
-

Friday: Defensive Byte Mapping with Dataclasses

- **Conceptual Focus:** Packaging raw values into clean, type-hinted object models (`dataclasses`).
-



Hands-On Assignment: Hardware Telemetry Parser

Build a robust Python program named `telemetry_parser.py` that connects to a virtual serial port, extracts framed packets, checks CRC-16, and logs outputs.

1. Requirements

- Define a packet frame structure: `SOP` (1 byte, `0xAA`), `Device_ID` (1 byte), `Voltage` (32-bit float), `Counter` (16-bit int), `CRC-16` (2 bytes), `EOP` (1 byte, `0xBB`).
- Implement a serial reader loop:
- Continually read bytes from the serial handle.
- Locate valid frames bounded by `SOP` and `EOP`.
- Validate the payload using a custom CRC-16 calculation function.
- If CRC passes, unpack components using the `struct` module and store them inside a type-hinted Python dataclass.
- Code must be fully type-hinted, checked with `mypy --strict`, and format-locked via Black.

2. File Deliverables

- `telemetry_parser.py`: Main parsing script containing connection and loops.
- `virtual_serial.py`: A simulated serial port utility that generates raw byte telemetry streams with randomized data corruptions.



Deliverables Checklist

- ☐ `telemetry_parser.py` (Mypy type-hinted, zero warnings)
- ☐ `virtual_serial.py` (Mock telemetry generator utility)
- ☐ `requirements.txt` (Environment libraries dependencies)

? Self-Check Questions

1. Why is setting an explicit read timeout critical inside serial loop connection handles?
 2. What do the character codes `<` and `>` represent inside Python `struct` format strings, and why are they mandatory for cross-target parsing?
 3. Explain the mechanical steps of locating a packet frame inside a raw continuous byte stream.
 4. How does a Cyclic Redundancy Check (CRC) identify bitwise data corruption in noisy transmission environments?
 5. Why are standard Python dataclasses superior to raw index tuples for representing unpacked telemetry values?
-

Stretch Task

Modify your telemetry parser to support **CSV telemetry recording**. Integrate logic to automatically write valid telemetry packets into a thread-safe CSV log file, rotating logs dynamically when file sizes exceed 1MB.

Motivation Checkpoint

Telemetry tracking is the heartbeat of spacecraft, automotive, and aeronautics control. During space missions (like Apollo or Mars Lander), data-logging scripts parse thousands of binary frames a second. If a script lacks robust boundary scanning or CRC validation, a single bit flip can corrupt the metrics dashboard and blind flight controllers.

Trainer Review Rules

- Verify type-safety: Executing `mypy --strict telemetry_parser.py` must complete with 0 alerts.
- Verify fault-tolerance: Passing malformed packets containing invalid CRCs or truncated lengths must be caught and logged cleanly without throwing unhandled Python parser exceptions.

Week 03: MicroPython & Microcontroller Prototyping

Weekly Goal

Master the execution context of MicroPython running on edge microcontroller hardware (ESP32/RP2040), control physical peripherals (GPIO, ADC, PWM), write interrupt service routines defensively, and automate host-to-target scripts deployment.

Weekly Outcome

By the end of this week, you will be able to program microcontrollers using MicroPython, configure hardware register abstractions, write clean hardware interrupt routines, manage edge data storage, and deploy automated flashing scripts using command-line wrappers.

17 Session Breakdown

Monday: The MicroPython Paradigm

- **Conceptual Focus:** Python in silicon.
- How MicroPython wraps a C-based interpreter that fits inside microcontrollers with limited SRAM (typically < 512KB).
- The boot sequence: `boot.py` (runs once at power-up) and `main.py` (runs the main loop).
- Interacting with the MicroPython REPL (Read-Eval-Print Loop) dynamically over USB serial.
- **Hands-on Experiment:** Auditing standard modules and memory inside the MicroPython shell:
````python import gc import machine`

# Check free RAM in bytes `print(f"Free memory: {gc.mem_free()} bytes")` ```

---

### Tuesday: GPIO & Peripheral Control

- **Conceptual Focus:** Reading and writing signals.
- Controlling pins using `machine.Pin`.
- **Digital Output:** Driving pins High (3.3V) or Low (0V).
- **Digital Input:** Reading button states with pull-up/pull-down resistor configurations.
- **Hands-on Exercise:** Define and blink an LED pin: ````python from machine import Pin import time`

```
Configure Pin 2 as output led = Pin(2, Pin.OUT)

Toggle pin state for _ in range(5): led.value(1) time.sleep_ms(500) led.value(0) time.sleep_ms(500)
...

```

---

### Wednesday: Analog Interfacing & PWM (ADC / PWM)

- **Conceptual Focus:** Continuous wave manipulation.
  - **Analog-to-Digital Converter (ADC):** Reading sensor voltages (e.g. temperature sensors, potentiometers) and mapping them to digital ranges (e.g., 12-bit range 0-4095).
  - **Pulse-Width Modulation (PWM):** Controlling LED brightness or motor speeds by adjusting duty cycles.
  - **Hands-on Exercise:** Read analog voltages and write PWM outputs dynamically.
- 

### Thursday: Defensive Hardware Interrupts (IRQs)

- **Conceptual Focus:** Asynchronous physical events.
- **Interrupt Service Routine (ISR):** A function triggered immediately by a hardware pin transition (e.g., button press).
- **Critical Rules for ISRs in MicroPython:**
  - Keep it short! Long routines block other system operations.
  - **No Allocations:** Banish standard list creation or string formatting inside ISRs (allocating memory triggers errors inside interrupt contexts).
  - Use `micropython.alloc_emergency_exception_buf` to capture ISR crash diagnostics.
- **Hands-on Exercise:** Configure a safe, low-latency button interrupt: `python import micropython from machine import Pin`

```
Allocate emergency buffer for interrupts micropython.alloc_emergency_exception_buf(100)

btn_pressed = False

def button_handler(pin): global btn_pressed btn_pressed = True # Short operation, zero allocations

btn = Pin(12, Pin.IN, Pin.PULL_UP) btn.irq(trigger=Pin.IRQ_FALLING, handler=button_handler)

```

---

### Friday: Host-Target Deployment Automation

- **Conceptual Focus:** Automating code deployments.
  - Bypassing manual copy-pasting. Using Python host tools ( `ampy` , `rshell` ) to upload files dynamically.
-

## Hands-On Assignment: Edge Data-Logger Node

---

Design and write a complete MicroPython application ( `main.py` ) that acts as an **Edge Temperature Data-Logger** with hardware interrupt triggers.

### 1. Requirements

- Periodically read temperature from an analog sensor using ADC every 2 seconds.
- If temperature exceeds a ceiling threshold (e.g. 35°C), trigger a flashing LED alert using PWM.
- Provide an emergency stop interrupt: pressing a button must trigger an ISR that halts logging and safely de-energizes the LED immediately.
- Log all sensor records (timestamp, value, alert\_state) into a local CSV file on the target's flash memory, checking that files are closed cleanly to prevent flash wear.

### 2. File Deliverables

- `boot.py` : Configures target setup.
  - `main.c` (MicroPython `main.py` ): The core sensor and interrupt logging loop.
  - `deploy.py` : A host-side Python automation script that automatically flashes the target files over serial USB.
- 

## Deliverables Checklist

---

- ☐ `boot.py` (Initialization code)
  - ☐ `main.py` (Hardened interrupt and logging loops)
  - ☐ `deploy.py` (Automated flashing python script)
- 

## ? Self-Check Questions

---

1. Why is the available memory (RAM) in MicroPython significantly smaller than in standard desktop CPython?
2. Explain why dynamic memory allocations (e.g. string formatting) are prohibited inside MicroPython hardware Interrupt Service Routines (ISRs).
3. How does `machine.Pin.PULL_UP` prevent digital input floating pin states?
4. What does the emergency exception buffer `micropython.alloc_emergency_exception_buf()` resolve when debugging ISRs?
5. How can you automate file transfers to a MicroPython microcontroller using command-line wrappers?

---

## Stretch Task

---

Implement an **asynchronous logger** using MicroPython's `uasyncio` module. Structure your logging code to execute as concurrent async tasks (one task reading sensors, one task flashing LEDs, and one task monitoring interrupts), avoiding blocking loops entirely.

---

## Motivation Checkpoint

---

IoT edge nodes (like smart home sensors, agricultural monitors, and predictive maintenance collars) must run for years on small batteries. MicroPython allows rapid prototyping of these systems. Writing defensive, low-latency, interrupt-driven edge code ensures that devices stay online without crashing or draining battery reserves.

---

## Trainer Review Rules

---

- Verify ISR execution: Pressing the emergency button must trigger an immediate de-energization loop within microseconds.
- Verify memory discipline: Run garbage collection checks inside the loops to verify that active operations do not trigger memory leaks or excessive garbage collection sweeps.

# Week 04: Instrumentation Control (PyVISA & SCPI)

---

## Weekly Goal

---

Master the SCPI (Standard Commands for Programmable Instruments) standard, interface with lab hardware dynamically using PyVISA, and write robust, reusable device drivers to control instruments (oscilloscopes, power supplies, load units).

## Weekly Outcome

---

By the end of this week, you will be able to map active VISA resources, write a complete object-oriented driver wrapper for a programmable DC power supply, execute SCPI queries securely over raw socket and USB connections, and handle device errors gracefully.

---

## Session Breakdown

---

### Monday: The VISA Standard & PyVISA

- **Conceptual Focus:** The Virtual Instrument Software Architecture (VISA) standard.
- Interfacing with instruments across different physical interfaces (USB, Ethernet, GPIB, Serial) using a unified software API.
- Installing and configuring the PyVISA backend.
- Querying active VISA resources.
- **Hands-on Experiment:** Scan for active hardware interfaces: `python import pyvisa`

```
Initialize VISA resource manager rm = pyvisa.ResourceManager()
```

```
List all detected resources resources = rm.list_resources() print("Detected instruments:") for res in resources: print(f" -> {res}") ``
```

---

### Tuesday: The SCPI Command Standard

- **Conceptual Focus:** Standard Commands for Programmable Instruments.
- SCPI is a hierarchical tree-like text protocol.
- Common IEEE 488.2 commands:
  - `*IDN?` : Identification query (returns manufacturer, model, serial).

- `*RST` : Reset command (restores default safe factory configurations).
  - `*OPC?` : Operation Complete query (prevents command race conditions).
  - SCPI Syntax rules: colons (`:`), question marks (`?`), parameter spacing.
  - **Hands-on Exercise:** Trace SCPI command structures:  
`:SOURCE:VOLTAGE:LEVEL:IMMEDIATE:AMPLITUDE 5.0` (set voltage to 5V).
- 

## Wednesday: Device Driver Design Pattern

- **Conceptual Focus:** The Device Object Model (DOM).
- Why you must **never** hardcode raw SCPI strings directly in your test suites.
- Wrapping raw VISA write/query calls inside clean, type-hinted object-oriented wrapper classes.
- **Hands-on Exercise:** Build a basic VISA instrumentation wrapper: `python import pyvisa`

```
class PowerSupply:
 def __init__(self, visa_address: str):
 self.rm = pyvisa.ResourceManager()
 self.device = self.rm.open_resource(visa_address)
 # Set safe write/read timeouts
 self.device.timeout = 2000 # 2 seconds
```

```
 def reset(self) → None:
 self.device.write("*RST")

 def get_identity(self) → str:
 return str(self.device.query("*IDN?")).strip()

 def close(self) → None:
 self.device.close()
```

...

---

## Thursday: Raw TCP Socket Communications

- **Conceptual Focus:** Controlling Ethernet-connected instruments without VISA layers.
  - Interfacing directly via raw TCP socket programming (often on Port 5025).
  - **Hands-on Exercise:** Connect and query an instrument over raw sockets using Python's `socket` module.
- 

## Friday: Error Queues & Status Registers

- **Conceptual Focus:** Auditing device internal states.
- Querying system errors using `:SYSTEM:ERROR?`.

- The necessity of clearing device error queues before and after test runs.
- 

## Hands-On Assignment: Programmable Power Supply Driver

---

Design and implement a complete, fully type-annotated, and robust Python device driver class (`power_supply.py`) to wrap a programmable DC Power Supply.

### 1. Requirements

- Define a class `ProgrammablePowerSupply` utilizing PyVISA.
- Implement the following methods:
  - `__enter__` and `__exit__` context managers (resource protection).
  - `reset()` : Resets the instrument.
  - `set_voltage_limit(limit: float)` : Sets an absolute safe over-voltage protection limit.
  - `set_output(voltage: float, current: float)` : Sets output parameters, checking bounds defensively.
  - `enable_output(state: bool)` : Toggles the output relay.
  - `measure_voltage() → float` : Queries active measured output voltage.
  - `check_errors() → list[str]` : Queries the error queue until empty.
- All code must be strictly type-annotated and pass `mypy --strict`.

### 2. File Deliverables

- `power_supply.py` : The instrument driver class.
  - `mock_instrument.py` : A simulated SCPI socket server utility that mimics a real programmable power supply (accepts connection, processes SCPI strings, and returns mock values).
- 

## Deliverables Checklist

---

- [ ] `power_supply.py` (Mypy type-hinted, context managers)
  - [ ] `mock_instrument.py` (SCPI socket simulator)
  - [ ] `requirements.txt` (Environment libraries dependencies)
- 

## ? Self-Check Questions

---

1. What is the fundamental purpose of the Virtual Instrument Software Architecture (VISA) standard in engineering labs?

2. Explain the hierarchical tree structure of SCPI commands, and why short/long formats (e.g. `MEASure:VOLTage?`) are supported.
  3. Why are context managers (`__enter__` / `__exit__`) essential when designing instrumentation drivers?
  4. How does the SCPI command `*OPC?` prevent synchronization errors when sending slow configurations (like device resets)?
  5. How does raw socket interfacing bypass the need to install heavy NI-VISA vendor backend installations?
- 

## Stretch Task

---

Modify your Power Supply driver to support **ramping profiles**. Implement a method `ramp_voltage(target: float, step: float, delay_ms: int)` that increments the voltage gradually from the current level to the target level, validating status states at each step to prevent hardware voltage surges.

---

## Motivation Checkpoint

---

In automotive, battery, and medical HIL testing labs, test racks contain millions of dollars of hardware components. If an automated script sends an incorrect SCPI voltage parameter due to a driver typo or parsing error, it can physically incinerate the device-under-test. Writing robust, fully typed, and bounds-checked drivers is a mandatory engineering safety law.

---

## Trainer Review Rules

---

- Verify type-safety: Executing `mypy --strict power_supply.py` must complete with 0 alerts.
- Verify context management: Initiating a connection and intentionally triggering a Python crash must close the raw VISA socket cleanly, confirmed by socket port scans.

# Week 05: Test Framework Architecture (pytest & Fixtures)

---

## Weekly Goal

---

Master the pytest testing framework, leverage the power of dependency-injection fixtures to isolate hardware states, build parameterized test matrixes, and implement defensive hardware timeout and retry mechanisms.

## Weekly Outcome

---

By the end of this week, you will be able to construct stateless test suites, write scoped setup/teardown fixtures that manage physical instrument lifetimes, write parameterized test cases to verify systems across varying bounds, and generate structured XML test results.

---

## Session Breakdown

---

### Monday: The Pytest Architecture

- **Conceptual Focus:** Professional testing paradigms.
  - Why standard `unittest` is archaic (too boilerplate, relies on heavy class inheritance).
  - Pytest's functional testing approach.
  - Test discovery rules (looks for files matching `test_*.py` and functions matching `test_*`).
  - Basic assertions and pytest command-line switches (`-v` verbose, `-s` show stdout, `-x` exit on first failure).
  - **Hands-on Experiment:** Write a simple test file and run it using pytest.
- 

### Tuesday: The Power of Fixtures

- **Conceptual Focus:** Stateless test isolation.
- What is a **Fixture**? A helper function that executes setups (pre-test configuration) and teardowns (post-test cleanup).
- **Fixture Scoping:** Controlling fixture lifetimes:
  - `function` (default): Runs once for every test case.
  - `module`: Runs once per test file.

- `session`: Runs once per test suite execution.

- **The Yield Keyword:** Splitting setup and teardown phases.

- **Hands-on Exercise:** Build a fixture managing a mock connection: ``python import pytest

```
@pytest.fixture(scope="function") def database_connection(): # Setup phase conn = "ESTABLISHED"
print("\n[SETUP] DB Connection opened.") yield conn # Teardown phase print("\n[TEARDOWN] DB
Connection closed.")
```

```
def test_query(database_connection): assert database_connection == "ESTABLISHED" print(" ->
Running test query.") `` Run with: pytest -s -v test_file.py` and inspect the setup/teardown
sequence.
```

---

## Wednesday: Hardware Fixtures & Resource Protection

- **Conceptual Focus:** Injecting hardware drivers cleanly.
  - Managing lab equipment connections safely.
  - Session-scoped fixtures establish hardware handles once, while function-scoped fixtures reset instrument outputs to safe baselines before every test.
  - **Hands-on Exercise:** Write a session-scoped fixture that opens a VISA instrument and a function-scoped fixture that resets it to factory defaults.
- 

## Thursday: Parameterized Test Matrixes

- **Conceptual Focus:** High-scale verification matrices.
- Avoid duplicate test code. Using `@pytest.mark.parametrize` to run a single test block across a list of varying inputs.
- **Hands-on Exercise:** Parameterize a voltage check: ``python import pytest

```
@pytest.mark.parametrize("voltage, current", [(1.2, 0.1), (3.3, 0.5), (5.0, 1.0)]) def
test_voltage_limits(voltage, current): print(f"Testing target power state: {voltage}V @ {current}A")
assert voltage <= 5.0 ``
```

---

## Friday: Hardware Retries & Timeouts

- **Conceptual Focus:** Dealing with intermittent hardware noise.
  - Hardware tests occasionally fail due to minor signal noise.
  - Implementing retry decorators and managing execution timeouts defensively.
-

# Hands-On Assignment: Hardware Characterization Test Suite

---

Design and implement a modular pytest test suite ( `test_hardware.py` ) that audits a target device's electrical power states.

## 1. Requirements

- Wrap the `ProgrammablePowerSupply` driver (from Week 4) inside a pytest fixture.
- Structuring the fixtures:
  - `instrument_session` (session-scoped): Establishes the VISA connection and yields the driver object. Teardown must disable output and close VISA resource.
  - `power_rail` (function-scoped): Accepts `instrument_session` , resets it, sets standard voltage safety limits, and yields the driver.
- Write a parameterized test case `test_voltage_regulation` :
- Tests voltage settings from `1.0V` to `5.0V` in `0.5V` steps.
- In each test: sets power supply voltage, sleeps for 100ms, measures active voltage, and asserts that the measured voltage is within a 2% margin of the commanded voltage.
- Code must be fully type-hinted and pass `mypy --strict` .

## 2. File Deliverables

- `test_hardware.py` : The pytest file.
  - `conftest.py` : Shared fixtures file (pytest standard).
  - `pytest.ini` : Framework configuration file.
- 

## Deliverables Checklist

---

- ☐ `conftest.py` (Session and function fixtures)
  - ☐ `test_hardware.py` (Parameterized test cases)
  - ☐ `pytest.ini` (Config files)
  - ☐ `requirements.txt` (Environment libraries dependencies)
- 

## Self-Check Questions

---

1. Why is global state sharing considered highly dangerous when writing automated hardware testing suites?
2. Explain how the `yield` statement partitions setup and teardown stages inside a pytest fixture.

3. What are the mechanical differences between a `session`-scoped fixture and a `function`-scoped fixture?
  4. How does the decorator `@pytest.mark.parametrize` optimize code reuse when testing boundaries?
  5. Why should a hardware test case incorporate explicit test timeouts, and how do you achieve this in pytest?
- 

## Stretch Task

---

Write a custom **pytest plugin hook** (inside `conftest.py`) that captures test execution statistics. If a test case fails, intercept the hook, query the simulated instrument's error queue automatically, and append the device error log directly into the pytest failure report.

---

## Motivation Checkpoint

---

Automated test coverage is the line between quality hardware and catastrophic failure. Hardware teams at companies like Tesla and SpaceX run thousands of parameterized test scenarios dynamically against ECUs and batteries. Structuring tests stateless and isolating hardware resources via fixtures prevents cascading test failures and guarantees clean test execution data.

---

## Trainer Review Rules

---

- Verify type-safety: Executing `mypy --strict test_hardware.py conftest.py` must complete with 0 alerts.
- Verify clean teardowns: Force a test to fail (e.g. inject an assert failure). The teardown code inside fixtures must still execute, cut the output power rails of the simulated supply, and close the sockets cleanly.

# Week 06: Hardware-in-the-Loop (HIL) & Fault Injection

---

## Weekly Goal

---

Master the principles of Hardware-in-the-Loop (HIL) testing, control physical relay boards to dynamically manipulate electrical signals, and build automated fault injection scenarios to stress-test device safety responses.

## Weekly Outcome

---

By the end of this week, you will be able to construct HIL test loops, write driver layers to control multi-channel USB/Ethernet relay boards, automate physical fault injections (signal short-circuits, brownouts, power cuts), and monitor asynchronous target responses under load.

---

## Session Breakdown

---

### Monday: Hardware-in-the-Loop (HIL) Principles

- **Conceptual Focus:** Testing physical hardware in a closed loop.
  - In advanced systems, software cannot be tested in pure isolation on PC simulators. It must run on the actual target microcontroller.
  - **The HIL Concept:** The target microcontroller's pins are connected to a test rack containing automated testing instruments and relay boards. Python controls the test rack to simulate a real physical environment (sensors, signals, power grids).
  - closed-loop vs open-loop test architectures.
  - **Hands-on Experiment:** Trace block schematics of a standard HIL test loop connecting host scripts, simulated sensor buses (I2C/SPI), and physical target pins.
- 

### Tuesday: Relay Board Driver Design

- **Conceptual Focus:** Dynamically switching physical connections.
- What is a relay? An electromagnetically operated switch.
- Using multi-channel USB/Ethernet relay boards to connect and disconnect lines.

- De-energizing and energizing channels dynamically from Python via serial commands or raw socket controls.
- **Hands-on Exercise:** Build a clean class wrapping a standard USB relay controller: ``python import serial

```
class RelayController: def init(self, port: str): self.ser = serial.Serial(port, 9600, timeout=1.0)
```

```
def set_channel(self, channel: int, state: bool) → None:
 # Standard hex protocol for a typical USB relay board
 cmd = bytearray([0xA0, channel, 0x01 if state else 0x00, 0x00])
 # Calculate verification sum
 cmd[3] = (cmd[0] + cmd[1] + cmd[2]) & 0xFF
 self.ser.write(cmd)

def close(self) → None:
 self.ser.close()
```

...

---

### Wednesday: Fault Injection Paradigms

- **Conceptual Focus:** Intentional system abuse.
- To verify that safety-critical firmware reacts correctly to physical hazards, we must inject faults during operation.
- **Core Fault Scenarios:**
  1. **Power Faults:** Brownouts (reducing voltage to critical boundaries), raw power cuts.
  2. **Bus Faults:** Disconnecting communication lines (SDA/SCL) mid-transaction.
  3. **Signal Faults:** Short-circuiting pins to Ground or 3.3V rails.
- **Hands-on Exercise:** Design the logical flow chart for an automated brownout injection test sequence.

---

### Thursday: Asynchronous Failure Monitoring

- **Conceptual Focus:** Spotting failures in real time.
- When a fault is injected, the target MCU might emit crash data over UART or trigger a physical GPIO alarm.
- Python must run asynchronous listening threads that watch for these indicators while the main test thread is executing fault steps.
- **Hands-on Exercise:** Write a Python thread that monitors a target serial output for crash messages ( "PANIC" , "ASSERT" ) during testing.

---

## Friday: Safe HIL Recovery Routines

- **Conceptual Focus:** Cleaning up after chaos.
  - If a fault injection test case aborts or crashes, the HIL rack must restore instruments and relays to safe, neutral, de-energized states to prevent equipment damage.
- 



## Hands-On Assignment: HIL Fault Injection Test Harness

---

Build an automated **HIL Fault Injection Test Harness** ( `hil_fault_harness.py` ) that simulates power anomalies and evaluates a virtual target MCU's recovery capabilities.

### 1. Requirements

- Wrap the `RelayController` and `ProgrammablePowerSupply` inside unified pytest fixtures.
- Implement a HIL test sequence that:
  - Powers up the virtual target device at `3.3V`.
  - Spawns an asynchronous monitoring thread listening to the target's serial telemetry channel.
  - Uses a relay channel to simulate an "I2C Bus Disconnect" fault (energize relay to disconnect SDA line).
  - Asserts that the target telemetry channel registers a `"BUS_ERROR"` warning within 500ms.
  - Recovers by de-energizing the relay (reconnecting SDA) and verifies that the target returns to `"SYSTEM_OK"` status.
- All code must be strictly type-annotated, checked with Mypy, and Black formatted.

### 2. File Deliverables

- `hil_fault_harness.py` : The pytest HIL execution file.
  - `virtual_target.py` : A socket utility that simulates a physical target microcontroller's telemetry port, responding to simulated power variations and relay signals.
- 



## Deliverables Checklist

---

- [ ] `hil_fault_harness.py` (Mypy type-hinted, context managers)
  - [ ] `virtual_target.py` (Telemetry socket simulator)
  - [ ] `requirements.txt` (Environment libraries dependencies)
-

## ? Self-Check Questions

---

1. Why is Hardware-in-the-Loop (HIL) testing critical in automotive and safety-critical industries?
  2. Explain how relay boards are leveraged in automated testing to simulate communication line cutoffs.
  3. What is a "Brownout Fault," and how does a programmable power supply simulate it under script control?
  4. Why must fault-monitoring threads execute asynchronously while HIL test cases execute?
  5. What are the core safety recovery steps a HIL test suite must execute if a test crashes midway?
- 

## Stretch Task

---

Implement an **automated recovery loop** that checks for target freeze conditions. If the virtual target stops emitting telemetry completely (deadlock or crash), trigger a relay to completely cycle the target's physical power line (Hard Reset), verifying that it boots back up cleanly.

---

## Motivation Checkpoint

---

Automated HIL fault-injection is the baseline requirement for obtaining safety certifications (like ISO 26262 for automotive ECUs). Companies like Boeing or Mercedes-Benz run millions of automated fault-injection cycles on test rigs, simulating engine failures, sensor outages, and battery short circuits to verify that safety-critical control loops protect human passengers.

---

## Trainer Review Rules

---

- Verify type-safety: Executing `mypy --strict hil_fault_harness.py` must complete with 0 alerts.
- The test execution log must show that all asynchronous threads are terminated cleanly. No zombie threads or unclosed socket connections are permitted.

# Week 07: Firmware Analysis & Binary Parsing

---

## Weekly Goal

---

Master the structure of compiled firmware files (Executable and Linkable Format - ELF), write Python tools to parse symbols and sections using the `pyelftools` library, and build automated static memory audits to prevent microcontroller stack-to-heap overlaps.

## Weekly Outcome

---

By the end of this week, you will be able to dissect ELF headers, map memory section sizes (`.text`, `.data`, `.bss`), extract specific symbol memory offsets, parse compiler map files, and build automated pre-flashing validation gates.

---

## Session Breakdown

---

### Monday: The ELF Layout & Header Parser

- **Conceptual Focus:** The Executable and Linkable Format (ELF) structure.
- Standard segments in compiled embedded binaries:
  - **ELF Header:** Identifies target architecture (ARM, RISC-V, x86), endianness, and entry address.
  - **Section Header Table:** Maps raw binary segments (`.text`, `.rodata`, `.data`, `.bss`).
  - **Symbol Table** (`.symtab`): Contains names and memory addresses of all functions and global variables.
- **Hands-on Experiment:** Write a Python script to parse basic ELF header info using `pyelftools`:

```
```python from elftools.elf.elffile import ELFFile
```

```
with open("firmware.elf", "rb") as f: elf = ELFFile(f) print("ELF Header Details:") print(f" -> Architecture: {elf.get_machine_arch()}") print(f" -> Endianness: {elf.little_endian}") ```
```

Tuesday: Mapping Memory Sections

- **Conceptual Focus:** Auditing static memory footprint.
- Microcontrollers have highly constrained Flash (typically < 1MB) and SRAM (typically < 128KB).
- **.text:** Compiled code size (consumes Flash).

- **.data / .bss:** Static and global variables (consumes SRAM).
- Why test automation must audit these sizes to prevent hardware out-of-memory crashes.
- **Hands-on Exercise:** Extract and print the sizes of all segments: `python from elftools.elf.elffile import ELFFile`

```
with open("firmware.elf", "rb") as f: elf = ELFFile(f) for section in elf.iter_sections(): print(f"Section: {section.name:<10} | Size: {section['sh_size']:<6} bytes | Addr: {hex(section['sh_addr'])}")
```

Wednesday: Extracting Symbols & Offsets

- **Conceptual Focus:** Dynamic symbol parsing.
 - Locating specific variables in memory dynamically.
 - Automating debugging: mapping symbol addresses to GDB breakpoints or checking variables during HIL runtime memory inspections.
 - **Hands-on Exercise:** Write a Python function that returns the hex address of any given function symbol name by iterating over the ELF `.symtab` section.
-

Thursday: Parsing Compiler Map Files

- **Conceptual Focus:** Extracting allocation summaries from text logs.
 - Most compilers (GCC) output detailed `.map` files during builds.
 - Using regex in Python to scan map files and identify massive global arrays that consume SRAM.
-

Friday: Pre-Flashing Integrity Validation

- **Conceptual Focus:** Designing CI/CD validation gates.
 - Automated testing checks: Block the flashing process immediately if the static SRAM allocation exceeds 90% of physical microcontroller limits, preventing bricked hardware.
-



Hands-On Assignment: Automated Memory Footprint Auditor

Design and build a Python command-line utility named `firmware_auditor.py` that parses compiled ELF binaries, audits segment allocations, and checks limits.

1. Requirements

- Provide a CLI interface that accepts: `--elf` (path to compiled firmware file), `--flash-limit` (Flash size limit in bytes), and `--sram-limit` (SRAM size limit in bytes).
- The script must:

- Parse the ELF file sections using `pyelftools`.
- Compute Total Flash Usage (`.text` + `.data` + `.rodata`) and Total SRAM Usage (`.data` + `.bss`).
- Validate these sizes against the parsed limits.
- If size allocations exceed 90%, print a warnings list. If they exceed 100%, print a fatal error message and exit with return code `-1`.
- All code must be strictly type-annotated, checked with Mypy, and Black formatted.

2. File Deliverables

- `firmware_auditor.py` : The static memory analysis script.
- `test_workspace/` : Directory containing dynamic mockup ELF binary files for test runs.

Deliverables Checklist

- [] `firmware_auditor.py` (Mypy type-hinted, zero warnings)
- [] `test_workspace/` (C compile mockups)
- [] `requirements.txt` (Environment libraries dependencies)

? Self-Check Questions

1. What are the differences between relocatable object files and Executable and Linkable Format (ELF) structures?
2. How does the size of the `.bss` section in an ELF file differ from the actual size it occupies on the physical disk?
3. Explain how static memory allocations (`.data` and `.bss`) can trigger hardware heap/stack overlaps at microcontroller runtime.
4. Why is it useful to extract function symbols address offsets dynamically using Python in automated test environments?
5. What does a compiler `.map` file record, and how does regex help parse it?

Stretch Task

Implement an **automated symbol address hook**. Expand your tool so that it reads a target firmware ELF, extracts the exact memory address of a global telemetry array variable, and writes this address directly into a test configuration file to automate runtime memory injection assertions.



Motivation Checkpoint

In safety-critical avionics firmware development, software is strictly audited for SRAM allocation footprint. Exceeding stack/heap allocations can trigger memory overrides, causing non-deterministic crashes mid-flight. Pre-flashing ELF validation scripts are the first line of defense in hardware release pipelines.



Trainer Review Rules

- Verify type-safety: Executing `mypy --strict firmware_auditor.py` must complete with 0 alerts.
- Verify limits verification: Inputting limits smaller than target ELF sections must exit with exact error codes and report the precise byte overflow calculations cleanly.

Week 08: High-Performance Telemetry

Data Logging

Weekly Goal

Master Python's threading execution model, implement thread-safe data queues to handle high-speed inputs, and build a high-performance telemetry data logger that streams metrics cleanly to disk without data losses.

Weekly Outcome

By the end of this week, you will be able to spawn concurrent consumer/producer threads, protect shared queues using `queue.Queue`, process incoming telemetry inputs asynchronously, write data portably to CSV or binary Parquet formats, and benchmark execution performance.

Session Breakdown

Monday: The Python Concurrency Model (GIL)

- **Conceptual Focus:** The Global Interpreter Lock (GIL).
- Why CPython restricts execution to a single thread at a time for CPU-bound tasks.
- Why threading is **highly effective for I/O-bound tasks** (such as serial reading, network listening, disk writing), where threads spend most of their time waiting for data.
- Spawning threads using the `threading` module.
- **Hands-on Experiment:** Spawn a parallel logging thread: `python import threading import time`

```
def log_worker() -> None: print("Logging thread started.") time.sleep(2) print("Logging thread complete.")
```

```
t = threading.Thread(target=log_worker) t.start() t.join() # Wait for thread to finish
```

Tuesday: Thread-Safe Queues (`queue.Queue`)

- **Conceptual Focus:** The Producer-Consumer pattern.
- In high-speed systems, reading data and writing data in the same thread blocks execution. While Python is writing to a slow disk, serial buffers overflow, leading to data loss.

- **The Solution:** A dedicated **Producer Thread** that reads incoming serial bytes and pushes them onto a queue, and a separate **Consumer Thread** that pops bytes off the queue and writes them to the disk.
- Using Python's built-in `queue.Queue` which handles internal lock synchronization automatically.
- **Hands-on Exercise:** Build a basic producer-consumer queue loop:

```
python import threading
import queue import time
```

```
data_queue: queue.Queue[int] = queue.Queue()
```

```
def producer() -> None: for i in range(5): data_queue.put(i) time.sleep(0.5)
```

```
def consumer() -> None: while True: item = data_queue.get() if item is None: # Exit signal break
print(f"Consumed: {item}") data_queue.task_done() ``
```

Wednesday: Async Telemetry Streaming

- **Conceptual Focus:** Structuring logging loops.
 - Protecting loops from blocking on I/O delays.
 - Creating clean exit signals (using `threading.Event` objects) to stop threads gracefully without locking.
-

Thursday: High-Performance Data Formats (CSV vs Parquet)

- **Conceptual Focus:** Structural storage.
 - **CSV:** Human-readable but massive, parsing is CPU-heavy.
 - **Parquet:** Columnar binary storage, compressed, lightning-fast reads/writes (utilizes Pandas).
 - **Hands-on Exercise:** Benchmarking the write speed and disk consumption of CSV logging vs binary Parquet mapping.
-

Friday: Queue Overflow & CPU Throttling

- **Conceptual Focus:** Defensive concurrency.
 - Managing queue limits (`maxsize`).
 - Preventing memory leaks when the consumer lags behind high-speed producers.
-



Hands-On Assignment: Concurrent Telemetry Logger

Design and implement a multi-threaded high-frequency **Telemetry Data Logger** (`concurrent_logger.py`) that handles high-speed inputs safely.

1. Requirements

- Implement a **Producer Thread** that reads simulated raw telemetry data from a socket or serial port and pushes it immediately onto a `queue.Queue`.
- Implement a **Consumer Thread** that pops data off the queue, processes validation parameters, and writes data portably to a CSV log.
- Ensure thread synchronization:
- Use a `threading.Event` to stop threads gracefully on terminal exits (Ctrl+C).
- Use timeouts inside `queue.get(timeout=1.0)` to check for exit signals periodically.
- All code must be strictly type-annotated, checked with Mypy, and Black formatted.

2. File Deliverables

- `concurrent_logger.py` : Main thread executor and queue manager.
- `mock_sensor.py` : Utility generating continuous, high-speed data feeds.

Deliverables Checklist

- [] `concurrent_logger.py` (Mypy type-hinted, safe threads)
- [] `mock_sensor.py` (High-speed socket data source)
- [] `requirements.txt` (Environment libraries dependencies)

? Self-Check Questions

1. Why does the Python Global Interpreter Lock (GIL) not hinder performance in I/O-bound telemetry readers?
 2. Explain how the Producer-Consumer pattern prevents serial port buffer overflow data losses.
 3. How does `threading.Event` optimize thread coordination compared to standard integer flag polling?
 4. What are the performance and storage size differences between CSV and Parquet file formats?
 5. How do you prevent thread locks (deadlocks) when multiple concurrent consumer threads share queues?
-

Stretch Task

Modify your telemetry logger to support **automatic data compression**. Integrate logic so that when a CSV file size exceeds 5MB, the consumer thread automatically spins up a background compression task (`zipfile` or `gzip`) to compress the old log file asynchronously while active logging continues uninterrupted.

Motivation Checkpoint

Automated test benches in flight simulation, battery evaluation, or turbine auditing generate millions of telemetry points per second. If a logging tool locks the execution thread for a fraction of a second during a disk flush, telemetry data is lost, rendering validation logs useless. Writing robust, thread-safe asynchronous loggers is a core systems requirement.

Trainer Review Rules

- Verify thread execution: The application must terminate cleanly on keyboard exits, joining and killing all threads. No zombie processes or locked sockets must remain.
- The logger must complete under high-frequency benchmarks with **zero data frame losses**, validated by matching sent sequence numbers against written log indexes.

Week 09: Dynamic CLI Tools & Metrics Reporting

Weekly Goal

Master the creation of professional command-line interface (CLI) utilities using the `click` library, parse automation test outputs dynamically, and auto-generate beautifully styled PDF/HTML reports complete with performance data plots.

Weekly Outcome

By the end of this week, you will be able to write complex nested CLI commands, validate arguments dynamically, process test execution results databases, generate Matplotlib analysis charts, and compile them into automated PDF metrics reports.

Session Breakdown

Monday: Advanced CLIs with `click`

- **Conceptual Focus:** Declaring structured command trees in Python.
- Why standard `argparse` scales poorly for complex CLI utilities that require nested subcommands (like `git add` vs `git commit`).
- Introduction to **Click (Command Line Interface Creation Kit)**.
- Using decorators to declare commands (`@click.command`), subcommands (`@click.group`), parameters (`@click.option`), and argument validators.
- **Hands-on Experiment:** Build a basic Click subcommand tool: `python import click`

```
@click.group() def cli() -> None: pass
```

```
@cli.command() @click.option("--port", default="/dev/ttyUSB0", help="Serial port address") def connect(port: str) -> None: click.echo(f"Establishing serial connection to {port}")
```

```
if name == "main": cli() ``
```

Tuesday: Click Types & Validation

- **Conceptual Focus:** Defensive CLI arguments.

- Click parameter types: `click.Path` (validates path existence), `click.Choice` (restricts arguments to specific options), `click.IntRange`.
- **Hands-on Exercise:** Build a secure file parsing interface: `python import click from pathlib import Path`

```
@click.command() @click.option( "--config", type=click.Path(exists=True, file_okay=True,
dir_okay=False, path_type=Path), required=True, help="Path to YAML test configuration file" ) def
run_test(config: Path) -> None: click.echo(f"Verified config path exists: {config}")
```

Wednesday: Plotting Performance Metrics (Matplotlib)

- **Conceptual Focus:** Visualizing test results data.
 - Converting log arrays into statistical plots.
 - Generating voltage regulation sweeps, temperature rise logs, and test execution duration distributions.
 - Exporting charts dynamically to temporary files.
 - **Hands-on Exercise:** Write a Python function that consumes a CSV log, generates a voltage-versus-time line graph, and saves it as a PNG image.
-

Thursday: Automated PDF Reports Generation

- **Conceptual Focus:** Generating executive summary documents.
 - Creating styled reports using HTML structures and compiling them directly to PDF using **Weasyprint** within Python.
 - Injecting dynamic inline CSS, building tables of results, and embedding generated Matplotlib charts.
 - **Hands-on Exercise:** Write a Python script that reads an HTML template, replaces placeholders with test statistics and chart images, and compiles it directly to a PDF.
-

Friday: Interactive CLI Progress Animations

- **Conceptual Focus:** Enhancing user experiences.
 - Using Click UI helpers to display dynamic loaders (`click.progressbar` , spinners, status logs) while long-running tests are executing.
-



Hands-On Assignment: Automated Test Report Compiler

Design and build a professional command-line utility named `report_compiler.py` using Click that processes CSV test log outputs and auto-generates a PDF report.

1. Requirements

- Create a Click CLI subcommand tree:
- `report_compiler.py parse --log <path>`: Scans CSV log data and prints a summary to the console.
- `report_compiler.py compile --log <path> --output <path>`: Generates Matplotlib voltage plots, builds a styled HTML file containing test pass/fail metrics tables, and compiles it via Weasyprint to a PDF report.
- Incorporate Click progress bars to show compilation status.
- All code must be strictly type-annotated, checked with Mypy, and Black formatted.

2. File Deliverables

- `report_compiler.py`: The main Click CLI and compiler script.
- `mock_test_results.csv`: A mockup test logs file.



Deliverables Checklist

- ☐ `report_compiler.py` (Mypy type-hinted, Click groups)
- ☐ `mock_test_results.csv` (CSV log file)
- ☐ `requirements.txt` (Environment libraries dependencies)

? Self-Check Questions

1. Why is the Click library superior to `argparse` when constructing nested subcommands pipelines?
 2. How does `click.Path(exists=True)` harden your command-line tools against runtime path errors?
 3. Explain the mechanism used to embed dynamically generated Matplotlib plots into an HTML report before PDF compilation.
 4. Why is the combination of HTML templates and Weasyprint preferred over raw PDF writing libraries (like ReportLab)?
 5. What role do CLI progress indicators play in professional developer experience?
-

Stretch Task

Implement an **automated database summary hook**. Expand your tool so that it automatically parses a JUnit XML test results file, extracts failure details, and includes a dynamically styled "Failures Callout Box" in the generated PDF report highlighting failing files, line numbers, and tracebacks.

Motivation Checkpoint

In automated manufacturing and silicon validation teams, test arrays run thousands of validation sweeps overnight. Directors and engineers cannot read millions of raw CSV log rows to check hardware health. Building automated CLI tools that compile logs, plot sweeps, and generate executive PDF reports enables rapid, clean engineering sign-offs.

Trainer Review Rules

- Verify CLI parameter security: Attempting to call the CLI with missing files or out-of-range limits must be intercepted by Click's default validators and exit cleanly without raising Python code exceptions.
- The compiled PDF must contain the embedded Matplotlib PNG charts cleanly aligned within page margins, without clipping or formatting overlaps.

Week 10: Python Code Quality, Typing, & Design Patterns

Weekly Goal

Harness the full power of Python's static typing engine (`mypy`), deploy essential enterprise-grade object-oriented design patterns (Singleton, Factory, DOM) for hardware abstraction, and automate style consistency using black and ruff.

Weekly Outcome

By the end of this week, you will be able to write 100% type-safe Python code under strict Mypy checks, apply structural design patterns to isolate hardware configurations, eliminate open-connection lockups using Singleton managers, and automate style checks.

Session Breakdown

Monday: Strict Static Typing with Mypy

- **Conceptual Focus:** Bridge-building between dynamic Python and static type safety.
 - Why standard Python dynamic execution leads to subtle bugs in large automation frameworks (e.g. passing a string value to a function expecting a float).
 - Using Mypy for static type validation.
 - Core typing signatures: `Union` , `Optional` , `Callable` , `Any` (and why `Any` is considered a code smell under strict rules).
 - Advanced typing: generics (`TypeVar`), protocols (`Protocol` for static interfaces mapping).
 - **Hands-on Experiment:** Auditing type mismatches at compile time: `bash # Run strict static type check on src mypy --strict src/`
-

Tuesday: The Singleton & Connection Manager

- **Conceptual Focus:** Managing hardware connection safety.
- **The Problem:** In large test suites, multiple tests might attempt to open connection sessions to the same physical instrument simultaneously, locking ports and triggering communication errors.

- **The Solution:** The **Singleton Pattern** ensures that a class has only one instance and provides a global access point to it.
- **Hands-on Exercise:** Implement a thread-safe Singleton Connection Manager: ``python import threading from typing import Optional

```
class VISAConnectionManager: _instance: Optional["VISAConnectionManager"] = None _lock:
threading.Lock = threading.Lock()
```

```
def __new__(cls) -> "VISAConnectionManager":
    with cls._lock:
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._init_manager()
        return cls._instance

def _init_manager(self) -> None:
    self.connection_handle = "VISA_ESTABLISHED"
    print("[SINGLETON] Unique Connection Manager initialized.")
```

...

Wednesday: The Factory Design Pattern

- **Conceptual Focus:** Swapping hardware targets dynamically.
- **The Problem:** Tests need to run against the real physical microcontroller on HIL benches, but developers must also run tests locally against simulated mock objects during offline debugging.
- **The Solution:** The **Factory Pattern** decouples object creation, allowing the framework to dynamically instantiate real or mock device drivers based on configuration inputs.
- **Hands-on Exercise:** Build a driver factory that instantiates either `PhysicalOscilloscope` or `MockOscilloscope` depending on a config file string.

Thursday: The Device Object Model (DOM)

- **Conceptual Focus:** Abstraction layer separation.
- Representing physical hardware units as discrete class objects that map low-level registers and communications commands to high-level methods, keeping test code clean and readable.

Friday: Code Cleanliness Automations (Black & Ruff)

- **Conceptual Focus:** Automating style reviews.
- Running `black` to format code uniformly.

- Using `ruff` as a high-speed linter and style auditor to identify dead imports, performance issues, and format violations automatically.
-

Hands-On Assignment: HARDENED Device Driver

Framework

Take a disorganized, untyped, and unmaintained legacy hardware automation testing suite and refactor it into a 100% type-safe, object-oriented framework.

1. Requirements

- Implement a thread-safe `InstrumentManager` using the Singleton pattern.
- Implement a `DeviceFactory` that dynamically instantiates device wrapper classes.
- Incorporate strict Mypy types across all classes—running `mypy --strict` must return **zero** errors or warnings.
- Automate code formatting: all files must compile cleanly and match `black` and `ruff` rules.

2. File Deliverables

- `manager.py`: Singleton connection manager.
 - `factory.py`: Decoupled device driver factory.
 - `drivers.py`: Base interfaces and specific instrument wrappers (using DOM).
 - `test_hardened.py`: Test cases utilizing the drivers.
-

Deliverables Checklist

- ☐ `manager.py` (Mypy type-hinted, thread-safe Singleton)
 - ☐ `factory.py` (Dynamic object creator)
 - ☐ `drivers.py` (DOM layout wrappers)
 - ☐ `requirements.txt` (Environment libraries dependencies)
-

Self-Check Questions

1. Why is the `Any` type annotation restricted under strict Mypy checks, and what type-safe alternatives exist?
2. How does the Singleton pattern prevent port conflict errors inside concurrent testing setups?

3. Explain how the Factory pattern helps you write HIL tests that can run seamlessly offline using mocked hardware.
 4. What is the distinction between a low-level register write command and a Device Object Model (DOM) method?
 5. How do Black and Ruff optimize developer efficiency in software engineering teams?
-

Stretch Task

Implement an **asynchronous connection supervisor**. Expand your Singleton Manager to incorporate a background monitoring thread that pings the target hardware connections dynamically every 5 seconds. If a connection is dropped, trigger automated hot-swapping recovery routines.

Motivation Checkpoint

Enterprise automation frameworks (at companies like Google, Apple, or Intel) contain millions of lines of code. Dynamic typing and chaotic structure architectures lead to unmaintainable codebases. Applying clean OOP patterns and type annotations keeps frameworks maintainable, safe, and easily scalable across multiple test platforms.

Trainer Review Rules

- Verify type safety: Executing `mypy --strict` must pass with 0 errors.
- The framework must demonstrate successful Singleton behavior: instantiating the connection manager multiple times in different parts of the test run must yield the exact same memory pointer address.

Week 11: CI/CD Pipelines & Execution Orchestration

Weekly Goal

Harness Continuous Integration (CI/CD) engines to orchestrate hardware automation suites automatically on code changes, generate and parse standard JUnit XML test reports, and automate status alerts.

Weekly Outcome

By the end of this week, you will be able to write robust CI/CD pipeline scripts (GitHub Actions YAML / Jenkinsfile), configure automated execution environments, parse pytest JUnit XML results databases, check coverage thresholds dynamically, and automate Slack alerts.

Session Breakdown

Monday: The CI/CD Hardware Pipeline

- **Conceptual Focus:** The automated hardware release flow.
 - In professional engineering, every commit triggers automated builds.
 - **The HIL Pipeline:** When a developer pushes C code to a repository, the CI/CD server triggers a pipeline that:
 1. Compiles the firmware.
 2. Runs static code audits.
 3. Deploys the firmware to a physical HIL test bench.
 4. Invokes Python pytest automation suites.
 5. Records and parses results.
 - **Hands-on Experiment:** Trace block maps of a CI/CD server connecting to a local physical HIL host runner.
-

Tuesday: JUnit XML Reporting

- **Conceptual Focus:** Standardizing test data.
- JUnit XML is the global standard format for representing test results.

- Configuring pytest to output JUnit XML databases: `bash pytest --junitxml=build/report.xml`
- Parsing XML structures in Python using the `xml.etree.ElementTree` library to count passes, failures, and execution times.
- **Hands-on Exercise:** Parse a test report dynamically and output metrics to the console: `python`
`import xml.etree.ElementTree as ET`

```
tree = ET.parse("report.xml") root = tree.getroot()
```

```
# Extract summary attributes tests = int(root.attrib.get("tests", 0)) failures =  
int(root.attrib.get("failures", 0)) print(f"Total Tests Executed: {tests} | Failures: {failures}") ``
```

Wednesday: GitHub Actions for Automation

- **Conceptual Focus:** Writing workflow scripts.
 - Understanding GitHub Actions runner structures.
 - Structuring steps: checking out code, caching dependencies, setting up Python environments, running pytest, and archiving artifact reports.
 - **Hands-on Exercise:** Build a basic `.github/workflows/test.yml` runner configuration: `yaml`
`name: Automated Automation Tests on: [push] jobs: test: runs-on: ubuntu-latest`
`steps: - uses: actions/checkout@v3 - name: Set up Python uses: actions/setup-`
`python@v4 with: python-version: '3.10' - name: Install dependencies run: | pip`
`install -r requirements.txt - name: Run test suite run: | pytest --junitxml=build/`
`report.xml`
-

Thursday: Pipeline Quality Gates

- **Conceptual Focus:** Enforcing baseline metrics.
 - Blocking merges if code coverage falls below 90%, if there are static analysis warnings, or if any core hardware test case fails.
-

Friday: Slack & Email Notification Webhooks

- **Conceptual Focus:** Keeping teams informed.
 - Sending automated HTTP POST requests containing styled Markdown results summaries directly to team Slack channels after build completes.
-

Hands-On Assignment: Automated HIL Pipeline Orchestrator

Design and implement a complete **Automated Test Orchestrator** script (`pipeline_orchestrator.py`) and a GitHub Actions workflow that executes tests and alerts on failures.

1. Requirements

- Write a Python script `pipeline_orchestrator.py` that:
- Spawns pytest as a subprocess to execute the test suite and output `report.xml`.
- Parses `report.xml` using `xml.etree.ElementTree`.
- Formats an executive Markdown build summary (e.g. Tests Run: 15, Passed: 14, Failures: 1).
- Simulates dispatching this report via an HTTP POST request to a Slack webhook.
- Write a `.github/workflows/hil.yml` file that orchestrates the execution dynamically.
- All code must be strictly type-annotated, checked with Mypy, and Black formatted.

2. File Deliverables

- `pipeline_orchestrator.py`: Main parsing and dispatch script.
- `hil.yml`: GitHub Actions pipeline YAML configuration.

Deliverables Checklist

- ☐ `pipeline_orchestrator.py` (Mypy type-hinted, XML parsing)
- ☐ `hil.yml` (Complete GitHub Action script)
- ☐ `requirements.txt` (Environment libraries dependencies)

Self-Check Questions

1. Why is JUnit XML the preferred standard format for continuous integration test outputs?
 2. How do you construct a pipeline step in GitHub Actions that only executes if previous steps failed (capturing crash logs)?
 3. What is a "Pipeline Quality Gate," and how does it protect main repository branches?
 4. Explain how webhooks utilize HTTP POST payloads to transmit test data directly to Slack.
 5. In enterprise environments, how do local self-hosted runners differ from cloud runners when accessing physical lab hardware?
-

Stretch Task

Modify your orchestrator script to **auto-parse tracebacks**. Expand the XML parser so that if a test case fails, it automatically extracts the failure's stack traceback details and appends a "Developer Debugging Callout" directly into the Slack notification payload.

Motivation Checkpoint

Continuous integration is the heart of professional Agile engineering. Software giants and automotive manufacturers trigger millions of automated pipeline jobs daily. Setting up self-executing pipelines with rigorous quality gates ensures that buggy or unsafe code is blocked from ever reaching production hardware systems.

Trainer Review Rules

- Verify type safety: Executing `mypy --strict pipeline_orchestrator.py` must complete with 0 alerts.
- The XML parser must handle empty or malformed XML files cleanly, raising custom structured error alerts instead of crashing with raw parsing tracebacks.

Week 12: Production-Grade Capstone HIL Framework Project

Weekly Goal

Assemble all your object-oriented design patterns, PyVISA instrumentation wrappers, thread-safe asynchronous logging queues, and dynamic JUnit XML reporting engines to build an enterprise-grade **HIL Test Automation Framework** from scratch.

Weekly Outcome

By the end of this week, you will be able to construct large, type-safe, modular Python frameworks, manage complex hardware setups and teardowns dynamically, trace concurrent exceptions defensively, parse and graph telemetry sweeps, and present your work inside a professional systems portfolio.

Session Breakdown

Monday: The System Architectural Blueprint

- **Conceptual Focus:** Architecture of an enterprise-grade HIL automation suite.
 - Structural directories: `drivers/` (instrument layers), `tests/` (stateless verification suites), `telemetry/` (queue logging threads), `utils/` (config and report compilers).
 - Design pattern checkpoints: instantiating instruments via Singleton managers, configuring platforms dynamically via Factories.
 - **Hands-on Experiment:** Map a detailed class architecture schema displaying hardware drivers relationships and test injection points.
-

Tuesday: Multi-Threaded Exception Handling

- **Conceptual Focus:** Preventing framework crashes.
 - In concurrent frameworks, if a background logging thread throws an exception, standard programs run in silence while the main test continues, leading to silent test validity failures.
 - **The Law:** All background threads must incorporate global exception catch blocks that report crashes directly back to the main test loop.
-

Wednesday: Advanced Diagnostic Logging

- **Conceptual Focus:** Generating clear engineering audit trails.
 - Why standard `print()` statements are banned in professional frameworks.
 - Configuring Python's built-in `logging` module.
 - Logging levels: `DEBUG` (register level detail), `INFO` (test steps), `WARNING` (retries), `ERROR` (hardware bounds violations).
 - Directing logs to both standard outputs and persistent log files.
-

Thursday: Optimization & Hardware Safety Audits

- **Conceptual Focus:** Protecting fragile instrumentation rigs.
 - Checking the framework's behavior during abort events (Ctrl+C, power failures).
 - Ensuring absolute de-energization loops execute cleanly, even under crash conditions, protecting expensive lab equipment.
-

Friday: Assembling the Systems Portfolio

- **Conceptual Focus:** Demonstrating your expertise to hiring managers.
 - Writing high-impact README files detailing architecture, Mypy type passes, and JUnit reports.
 - Preparing for technical systems-level interviews.
-



Hands-On Assignment: Enterprise HIL Test Automation Framework

Design and build a fully functional, fully-typed, and multi-threaded **Hardware-in-the-Loop Test Automation Framework** (named `hil-framework`) in pure Python.

1. Requirements

- Structure the project cleanly across modular directories (`drivers/`, `tests/`, `telemetry/`, `reports/`).
- Implement:
 - A thread-safe `InstrumentManager` (Singleton pattern) managing VISA connections.
 - High-reliability `PowerSupply` and `Multimeter` wrappers using context managers.
 - A pytest testing harness that parameters tests over a sweep of voltage targets, monitors telemetry asynchronously, and logs records to Parquet/CSV formats.

- An automated `report_compiler.py` that processes logs, generates performance graphs (Matplotlib), and compiles a styled PDF summary.
- Code must compile cleanly, pass Mypy strict type checks with **zero warnings**, and run under black/ruff guidelines.

2. File Deliverables

- `drivers/` : Instrument wrapper modules.
- `tests/` : Parameterized pytest scenarios.
- `telemetry/` : Thread-safe async logging modules.
- `reports/` : PDF compilation and CLI scripts.
- `Makefile` : Automates setup, test runs, linting, and PDF reporting.

Deliverables Checklist

- ☐ Complete `drivers/`, `tests/`, `telemetry/`, and `reports/` folder structures.
- ☐ `Makefile` (Automated targets `make setup`, `make test`, `make lint`, `make report`)
- ☐ `requirements.txt` (Environment libraries dependencies)

? Self-Check Questions

1. Why is a centralized Exception Supervisor critical when executing concurrent test threads?
2. How does configuring Python's `logging` module outperform standard inline `print()` statements in large test runs?
3. What are the mechanical steps required to guarantee that a physical HIL test rack enters a safe state if the main controller PC loses power?
4. Explain how you would decouple test logic from physical address configurations to ensure portability across different lab benches.
5. What are the essential portfolio elements required to demonstrate test framework design mastery to recruiters?

Stretch Task

Implement an **automated recovery loop** that checks for target freeze conditions. If the virtual target stops emitting telemetry completely (deadlock or crash), trigger a relay to completely cycle the target's physical power line (Hard Reset), verifying that it boots back up cleanly.



Motivation Checkpoint

Writing a custom test automation framework is the ultimate graduation test of an automation engineer. It requires combining hardware communications, thread safety, testing isolation, data parsing, and continuous integration pipeline management. Completing this project successfully demonstrates that you are ready to construct enterprise-grade automated systems.



Trainer Review Rules

- Verify type safety: Executing `mypy --strict` on your framework must pass with 0 errors.
- Verify recovery safety: Forcing a test execution crash must de-energize the simulated power supply immediately and close all VISA connections cleanly.